

UNIT -3

Constructors, Destructors, Inheritance:

Introduction to Constructors, Default Constructors, Parameterized Constructors, Copy Constructors
Multiple Constructors in a Class, Destructors.

Inheritance :Introduction to inheritance, Defining Derived Classes, Single Inheritance, Multiple Inheritance, Multi-Level Inheritance, Hierarchical Inheritance, Hybrid Inheritance.

□ Introduction to Constructors: C++ provides a special member function called the constructor which enables an object to initialize itself when it is created. □

Definition:- A constructor is a special member function whose task is to initialize the objects of its class. It is special because its name is the same name as the class name. The constructor is invoked whenever an object of its associated class is created. It is called constructor because it constructs the values of data members of the class.

A constructor is declared and defined as follows:

```
class integer
{
    int m,n;
    public:
    integer( );
    .....
    .....
};
integer :: integer( )
{
    m=0;
    n=0;
}

int main()
{ integer obj1;
    .....
    .....
}
```

integer obj1; => not only creates object obj1 but also initializes its data members m and n to zero. There is no need to write any statement to invoke the constructor or function.

CHARACTERISTICS OF CONSTRUCTOR

- They should be declared in the public section.
- They are invoked automatically when the objects are created.
- They do not have return type, not even void.
- They cannot be inherited, though a derived class can call the base class constructor.
- Like other c++ functions, they can have default arguments.
- Constructors cannot be virtual.

➤ We cannot refer to their addresses.



They make „implicit calls“ to the operators new and delete when memory allocation is required.

Constructors are of 3 types:

1. Default Constructor
2. Parameterized Constructor
3. Copy Constructor

1. Default Constructor: A constructor that accepts no parameters is called the **default constructor**.

```
#include<iostream.h>
#include<conio.h>
class item
{
    int m,n;
public:
    item()
    {
        m=10; n=20;
    }
    void put();
};
void item::put()
{
    cout<<m<<n;
}
void main()
{
    item t;
    t.put();
    getch();
}
```

2. Parameterized Constructors:-The constructors that take parameters are called parameterized constructors. #include<iostream.h>

```
class item
{
    int m,n;
public:
    item(int x, int y)
    {
        m=x;
        n=y;
    }
};
```

When a constructor has been parameterized, the object declaration statement such as item t; may not work. We must pass the initial values as arguments to the constructor function when an object is declared. This can be done in 2 ways: item t=item(10,20); //explicit call

item t(10,20); //implicit call

Eg:

```
#include<iostream.h>
#include<conio.h>
class item
{
int m,n;
public:
    item(int x,int y)
    {
        m=x; n=y;
    }
    void put();
};
void item::put()
{
    cout<<m<<n;
}
void main()
{
item t1(10,20);
item t2=item(20,30);
    t1.put();
    t2.put();
    getch();
}
```

3.Copy Constructor: A copy constructor is used to declare and initialize an object from another object. Eg:

```
item t2(t1);
or item
t2=t1;
```

1. The process of initializing through a copy constructor is known as copy initialization.
2. t2=t1 will not invoke copy constructor. t1 and t2 are objects, assigns the values of t1 to t2.
3. A copy constructor takes a reference to an object of the same class as itself as

```
an argument. #include<iostream.h>
class sample
{
int n;
public:
```

```

sample()
{ n=0; }
sample(int a)
{

    n=a;
}
sample(sample &x)
{
    n=x.n;
}
void display()
{ cout<<n;
}
}; void
main()
{ sample
A(100); sample
B(A); sample
C=A;
sample D;
    D=A;
    A.display();
    B.display();
    C.display();
    D.display();
}

```

Output: 100 100

100 100

Multiple Constructors in a Class: Multiple constructors can be declared in a class. There can be any number of constructors in a class.

class comple x

```

{
float real,img;
public:
    complex()//default constructor
    {
real=img=0;
    }
    complex(float r)//single parameter parameterized constructor
    {
real=img=r;
    }
}

```

```

complex(float r,float i) //two parameter parameterized constructor
{
real=r;img=i;
}
complex(complex&c)//copy constructor
{
real=c.real; img=c.img;

}
complex sum(complex c )
{

complex t;
t.real=real+c.real;
t.img=img+c.img;
return t;
}
void show()
{
If(img>0)
    cout<<real<<"+"i"<<img<<endl;
else
    {
    img=-img;
    cout<<real<<"-"i"<<img<<endl;
    }
}
};
void main()
{
complex c1(1,2);
complex c2(2,2);
compex c3;
c3=c1.sum(c3);
c3.show();

}

```

DESTRUCTORS: A destructor, is used to destroy the objects that have been created by a constructor.

Like a constructor, the destructor is a member function whose name is the same as the class name but is preceded by a tilde. Eg: `~item() { }`

1. A destructor never takes any argument nor does it return any value.

2. It will be invoked implicitly by the compiler upon exit from the program to clean up storage that is no longer accessible.
3. It is a good practice to declare destructors in a program since it releases memory space for future use.

```
#include<iostream>
using namespace std;
    class Marks
    {
public:
    int maths;
    int science;

//constructor Marks() {    cout <<
"Inside Constructor"<<endl;
    cout << "C++ Object created"<<endl;
    }

//Destructor
    ~Marks() {    cout << "Inside
Destructor"<<endl; cout << "C++ Object
destruced"<<endl;
    }
};

int main( )
{
    Marks m1;
    Marks m2;
    return 0; }
```

Output:

```
Inside Constructor
C++ Object created
Inside Constructor
C++ Object created
Inside Destructor
C++ Object destruced
Inside Destructor
C++ Object destruced
```

INHERITANCE: . The mechanism of deriving a new class from an old one is called **inheritance** or **derivation**. The old class is referred to as the **base class** and the new one is called the **derived class** or **sub class**. The derived class inherits some or all of the traits from the base class.

A class can also inherit properties from more than one class or from more than one level. Reusability is an important feature of OOP

A derived class can be defined by specifying its relationship with the base class in addition to its own details.

```
class derived-class-name : visibility-mode base-class-name
{
    .....
    .....
}
```

The **colon** indicates that the derived class name is derived from the base-class-name. the visibility mode is optional and if present, may be either **private** or **protected** or **public**. The default is private. Visibility mode specifies whether the features of the base class are privately derived or publicly derived.

```
class ABC : private XYZ           //private derivation
{
members of ABC;
};
```

```
class ABC:public XYZ
{
members of ABC;
};
//public derivation
```

```
class ABC:protected XYZ {

// protected derivation members of ABC;
};
class ABC:XYZ                //private by default
{
members of ABC;
};
```

When a base class is **privately inherited** by a derived class, public members of the base class can only be accessed by the member functions of the derived class. private members of base class are inaccessible to the objects of the derived class

When a base class is **protected inherited** by a derived class, public members of the base class can only be accessed by the member functions of the derived class. private members of base class are inaccessible to the objects of the derived class. If private members of base class are to be inherited to derived class then declare them as protected

When the base class is **publicly inherited**, public members of the base class is publicly inherited, public members of the base class become public members of the derived class and therefore they are accessible to the objects of the derived class. In both the cases, the private members are not inherited and therefore, the private members of a base class will never become the members of its derived class

In inheritance, some of the base class data elements and member functions are „inherited“ into the derived class. We can add our own data and member functions and thus extend the functionality of the base class. Inheritance, when used to modify and extend the capability of the existing classes, becomes a very powerful tool for incremental program development

Visibility of inherited members

Base class visibility	Derived class visibility		
	Public derivation	Private derivation	Protected derivation
Private →	Not inherited	Not inherited	Not inherited
Protected →	Protected	Private	Protected
Public →	Public	Private	Protected

Types of
Inheritance:

1. Single Inheritance
2. Multi level Inheritance
3. Multiple Inheritance
4. Hybrid inheritance
5. Hierarchical Inheritance.

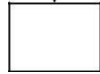
1.

SINGLE INHERITANCE: one derived class inherits from only one base class. It is the most simplest form of Inheritance.

A //Base class



B //Derived class



```
#include<iostream>
using namespace std;
class A
{
public:
int a,b;
void get()
{
cout<<"Enter any two Integer values"<<endl;
cin>>a>>b;
}
};

class B:public A
{ int c;
public:
void
add() {
c=a+b;
cout<<a<<"+"<<b<<"="<<c;
}
};

int main()
{
B b;
b.get();
b.add();
}
```

Output:

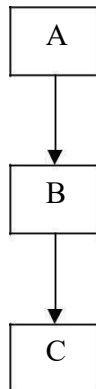
Enter any two Integer values

1 2

1+2=3

2.

MULTILEVEL INHERITANCE: In this type of inheritance the derived class inherits from a class, which in turn inherits from some other class. The Super class for one, is sub class for the other.



```
#include<iostream.h>
class A
{
public:
    int a,b;
    void get()
    {
        cout<<"Enter any two Integer values"<<endl;
        cin>>a>>b;
    }
};

class B:public A
{
public:
    int c; void
    add()
    { c=a+b;
    }
};

class C:public B
{
public:
    void show()
    {
        cout<<a<<"+"<<b<<"="<<c;
```

3.

```
    }  
}; int  
main()  
{  
    C c;  
    c.get();  
    c.add();  
    c.show();  
}
```

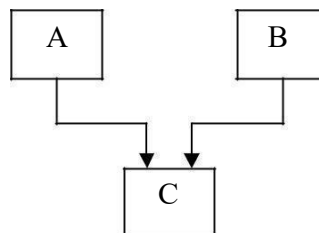
Output:

Enter any two Integer values

12 14

12+14=26

Multiple Inheritance: In this type of inheritance a single derived class may inherit from two or more than two base classes.



Syntax:

```
class D : visibility A,  
visibility B,... {  
.....  
}
```

```
#include<iostream.h>
```

```
class A
```

```
{  
public:  
int a; void getA()  
{  
    cout<<"Enter an Integer value"<<endl;  
    cin>>a;  
}
```

```
}; class
```

```
B {
```

```
public:  
    int b; void  
    getB()  
    {  
        cout<<"Enter an Integer value"<<endl; cin>>b;  
    }
```

```
};
```

```
class C:public A,public B
```

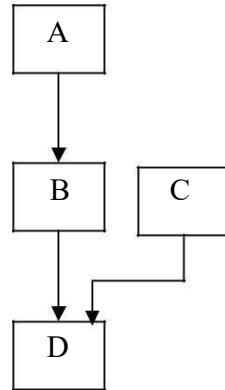
```
{  
public:  
int c; void add()  
    { c=a+b;
```

4.

```
        cout<<a<<"+"<<b<<"="<<c<<endl;
    }
}; int
main()
{ C obj;
obj.getA()
;
obj.getB()
;
obj.add();
}
Enter an Integer
value 12 Enter an
```

Integer value 13
12+13=25

4.Hybrid Inheritance: Hybrid inheritance is combination of two or more inheritances such as single,multiple,multilevel or Hierarchical inheritances.



```
#include<iostream.h>
class arithmetic
{ protected: int num1,
num2;
public:
    void getdata()
    {
        cout<<"For Addition:";
        cout<<"\nEnter the first number: "; cin>>num1;
        cout<<"\nEnter the second number: ";
        cin>>num2;
    }
};

class plus:public arithmetic
{ protected: int
sum;
public:
    void add()
    {
        sum=num1+num2;
    } };

class
minus
{
protected:
    int n1,n2,diff;
public:
    void sub() {
        cout<<"\nFor Subtraction:"; cout<<"\nEnter
the first number: "; cin>>n1;
        cout<<"\nEnter the second number: ";
```

```

        cin>>n2;
        diff=n1-n2;
    }
};
class result:public plus, public minus
{ public: void
display()
{
    cout<<"\nSum of "<<num1<<" and "<<num2<<"=
"<<sum; cout<<"\nDifference of "<<n1<<" and "<<n2<<"=
"<<diff; } }; int main() { result z;
    z.getdata();
    z.add();
    z.sub();
    z.display();
}

```

For Addition:

Enter the first number: 1

Enter the second number: 2

For Subtraction:

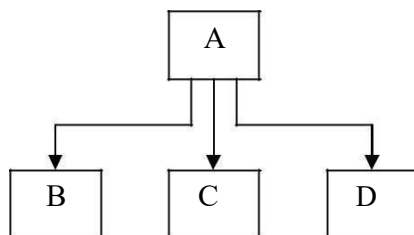
Enter the first number: 3

Enter the second number: 4

Sum of 1 and 2= 3

Difference of 3 and 4= -1

5.Hierarchical Inheritance:- Inheriting is a method of inheritance where one or more derived classes is derived from common base class.



```
#include<iostream.h>
```

```
class A //Base Class
```

```

{ public:
    int
    a,b;
    void getnumber()
    {

```

```

cout<<"\n\nEnter Number :\t"; cin>>a;
} };
class B : public A //Derived Class 1
{ public:
    void square()
    {
        getnumber(); //Call Base class property
        cout<<"\n\n\tSquare of the number :\t"<<(a*a); }
}; class C :public A //Derived
Class 2
{ public:
    void cube()
    {
        getnumber(); //Call Base class property
        cout<<"\n\n\tCube of the number ::: \t"<<(a*a*a); }
}; int
main()
{
    B    b1;    //b1 is object of Derived class 1
    b1.square(); //call member function of class
    B
    C    c1;    //c1 is object of Derived class 2
    c1.cube(); //call member function of class C
}
Enter Number : 2
    Square of the number : 4

Enter Number : 3

    Cube of the number ::: 27

```